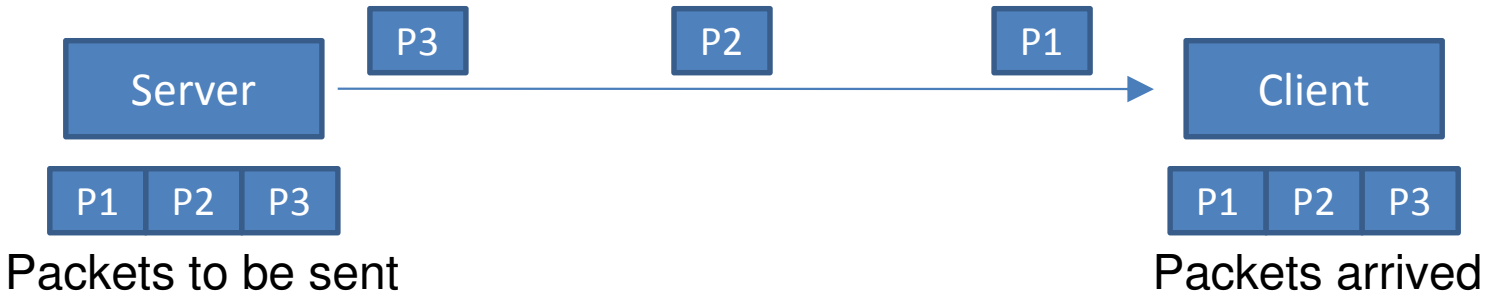
Java Client/Server

- Types of connections to server
- Sockets Client-Server Interaction
- Three Example Applications that uses Sockets
- Datagram Client-Server Interaction
- One Example Application that uses Datagrams

Prepared By: Dr.Osama Al-Haj Hassan

Types of Connections to Server

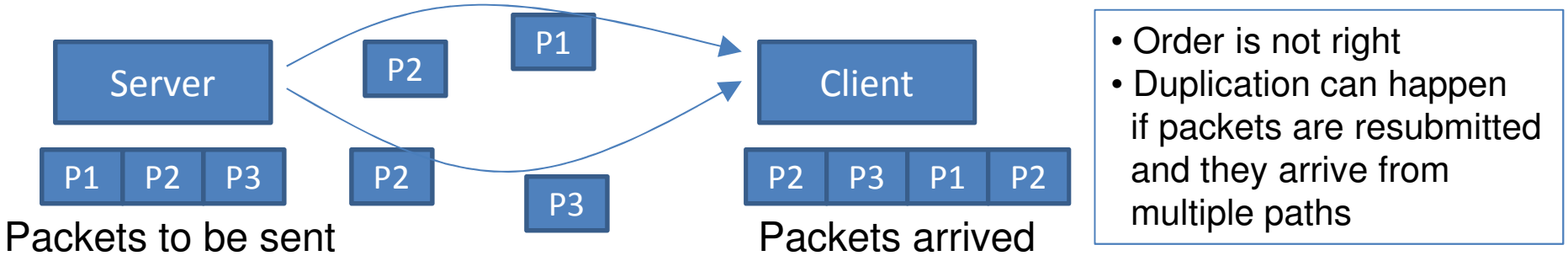
1) Connection-Oriented
   - One connection is established between client and server
   - Data flows in the communication channel in a continuous stream
   - Packets arrive in order
   - Low probability of packet loss
   - TCP (Transmission Control Protocol) is used for connection establishment and data transfer
   - In Java, this type of communication is programmed using “Sockets”



Types of Connections to Server

2) Connectionless

- Packets can go through different communication channels
- Packets are not guaranteed to arrive in order
- Packets can be lost or duplicated
- Protocol used is UDP (User Datagram Protocol)
- In Java, such type of communication is programmed using "Datagrams"
- This type of communication is used in applications where error checking and reliability of TCP is not needed.



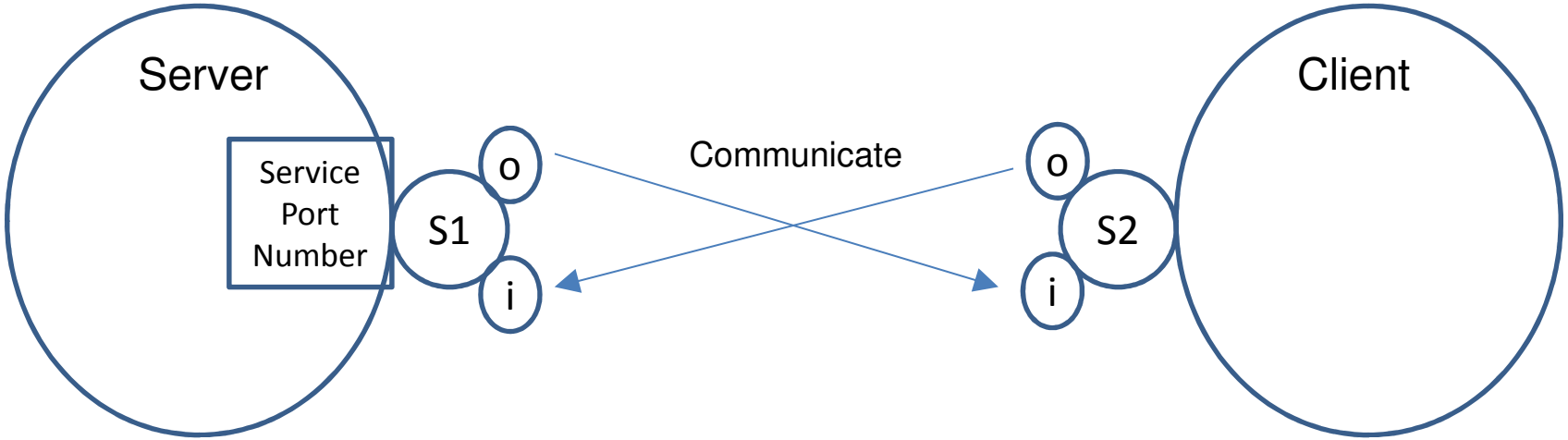
Note

- Connectionless applications are more efficient than Connection-oriented applications because connectionless applications do not generate the overhead of
  - No data loss
  - In order arrival guarantee

Simple Server/Client Communication

- So far, we have seen high level networking capabilities in Java.
- Now, we start with some low level networking capabilities.
- Connection Oriented Communication is done in java using
  - **Sockets**
- Connectionless oriented Communication is done in java using:
  - **Datagram**

Socket Client/Server Application



- Client uses server address and port number to get connected to server service
- Class “Socket” is used to establish communication between client and server
  - Client has its own socket (S2)
  - Client also has a corresponding socket at the server side (S1)
- Each socket has:
  - Input Stream (i) : to read data coming from destination
  - Output Stream (o) : to write data to destination
- Communication is done between input stream of one socket and the output stream of the other socket

Socket Client/Server Application

Steps the server needs to do to communicate with clients:

1) Establish Server

//This statement defines the server and attach the service to a given port number.

ServerSocket server = new ServerSocket(port_number, queue_size);

2) Waits for clients connections

// In this statement, server waits "indefinitely" for a client to connect to this server. The reference to communicate
// with client is his socket "clientSocket"

Socket clientSocket = server.accept();

3) Establish output stream to client

//This statement get the output stream to "clientSocket" which help server to send data to client

PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

4) Establish input stream to client

//This statement get the input stream to "clientSocket" which help server to receive data from client

BufferedReader input = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));

Socket Client/Server Application

Steps the clinet needs to do to communicate with server:

1) Connect to Server

//This statement allows client to connect to server at “serveraddress” using port number port_number
//The reference to communicate with server is the “clientSocket” at the server side

Socket serverSocket = new Socket(serverAddress, port);

2) Establish output stream to server

//This statement get the output stream to “clientSocket” at server side which help clients to send data to server

PrintWriter out = new PrintWriter(serverSocket.getOutputStream(), true);

3) Establish input stream to Server

//This statement get the input stream to “clientSocket” at server side which help client to receive data from server

BufferedReader input = new BufferedReader(new InputStreamReader(serverSocket.getInputStream()));

Socket Client/Server Application

- After Server gets Client Socket and Client gets his own Socket, communication starts between the two sockets using the input/output streams of sockets.
- Example1:
  - String message = input.readLine();
  - This statement reads data coming from destination
- Example2:
  - String message = “Send this text”;
  - out.println(message);
  - This statement sends “message” to destination

Socket Client –Server Applications

- Application1:
  - Server.java: server waits for a client to connect and then it sends “Hi There” to client
  - Client.java: receives message “Hi There” from server and displays it.

SocketServer.java

```
import java.io.*;
import java.net.*;

public class SocketServer
{
    public static void main(String[] args) throws IOException
    {
        ServerSocket listener = new ServerSocket(9090);
        System.out.println("Server is running");
        try
        {
                while (true)
                {
                    Socket socket = listener.accept();
                    try
                    {
                        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
                        out.println("Hi There");
                    }
                    finally
                    {
                        socket.close();
                    }
                }
        }
        finally
        {
                listener.close();
        }
    }
}
```

Create a server on port 9090

Loop to handle client after client indefinitely

This statement blocks server and let server keep waiting for a new client. When client connects, its socket is stored at "socket".

Get outputstream of client. This allows server to send something for client

Send something to client

Close client socket

Close server socket

true, means "AutoFlush is enabled". This means that anything written should be sent immediately to receiver instead of accumulating messages before sending them at once

SocketClient.java

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.Socket;

import javax.swing.JOptionPane;

public class SocketClient
{
   public static void main(String[] args) throws IOException
   {
      String serverAddress = JOptionPane.showInputDialog("Enter Server Name");
      String serverPort = JOptionPane.showInputDialog("Enter Port Number");
      int port = Integer.parseInt(serverPort);
      Socket s = new Socket(serverAddress, port);
      BufferedReader input = new BufferedReader(new InputStreamReader(s.getInputStream()));
      String answer = input.readLine();
      JOptionPane.showMessageDialog(null, answer);
      System.exit(0);
   }
}
```

Ask client for server and port he wants to connect to

Define socket that connects to server

Retrieve input stream of server. This allows client to receive from server.

Receive something from server.

Socket Client –Server Applications

- Application2:
  - Server_1.java: server waits for a client to connect and then it receives messages from client and send them back to him. If message equals “#” server disconnects connection with client.
  - Client_1.java: Keeps sending messages to server and receive them back

SocketServer_1.java

```
import java.io.*;
import java.net.*;

public class SocketServer_1
{
   public static void main(String[] args) throws IOException
   {
      ServerSocket listener = new ServerSocket(9090);
      System.out.println("Server is running");
      while (true)
      {
         Socket socket = listener.accept();
         PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
         BufferedReader input = new BufferedReader(new InputStreamReader(socket.getInputStream()));
         String msg="";
         while(true)
         {
            msg = input.readLine();
            if (msg.equals("#"))
            {
               out.println("From Server: terminatin connection");
               break;
            }
            else out.println("From Server: "+msg);
         }
         out.close();
         input.close();
         socket.close();
      }
   }
}
```

Get input stream of client. This allows server to receive something from client

Get output stream of client. This allows server to send something to client

A loop that lets server keep interacting with the same client

Server receives message from client and send it back to client. If server receives message "#" server closes connection with client

SocketClient_1.java

```
import java.io.*;
import java.net.Socket;
import javax.swing.JOptionPane;

public class SocketClient_1
{
   public static void main(String[] args) throws IOException
   {
      String serverAddress = JOptionPane.showInputDialog("Enter Server Name");
      String serverPort = JOptionPane.showInputDialog("Enter Port Number");
      int port = Integer.parseInt(serverPort);
      Socket s = new Socket(serverAddress, port);
      PrintWriter out = new PrintWriter(s.getOutputStream(), true);
      BufferedReader input = new BufferedReader(new InputStreamReader(s.getInputStream()));
      String msg="";
      while(true)
      {
         String toServer = JOptionPane.showInputDialog(null,"Send something to server");
         out.println(toServer);
         msg = input.readLine();
         System.out.println(msg);
         if (toServer.equals("#")) break;
      }
      out.close();
      input.close();
      s.close();
      System.exit(0);
   }
}
```

Retrieve input and output stream of server. This allows client to send/receive from server.

Loop that asks user write a message. The message is sent to server. Then client receives the message from server.

If clients sent "#" to server, this means he wants to disconnect

Socket Client –Server Applications

- Application3:
  - Server_2.java: server waits for two clients to connect server receives message from one client and sends it to the other client. This continues until either client sends “#” to server.
  - Client_2.java: Keep sending message to the other client through server and receives message from the other client through server.

SocketServer_2.java

```
import java.io.*;
import java.net.*;
public class SocketServer_2{
   public static void main(String[] args) throws IOException{
      ServerSocket listener = new ServerSocket(9090);
      System.out.println("Server is running");
      while (true){
         Socket socket1 = listener.accept();
         PrintWriter out1 = new PrintWriter(socket1.getOutputStream(), true);
        BufferedReader input1 = new BufferedReader(new InputStreamReader(socket1.getInputStream()));
         out1.println("Connected to server");
         Socket socket2 = listener.accept();
         PrintWriter out2 = new PrintWriter(socket2.getOutputStream(), true);
         BufferedReader input2 = new BufferedReader(new InputStreamReader(socket2.getInputStream()));
         out2.println("Connected to server");
         String msg1="";String msg2="";
         while(true){
            msg1 = input1.readLine();
            if(!msg1.equals("")){
               out2.println(msg1);
            }
            msg2 = input2.readLine();
            if(!msg2.equals("")){
               out1.println(msg2);
            }
            if (msg1.equals("#") || msg2.equals("#")) break;
         }
         out1.close();input1.close();socket1.close();out2.close();input2.close();socket2.close();
      }
   }
}
```

Wait until client 1 connects, and then get the input stream and output stream of client 1

Wait until client 2 connects, and get I/O streams

A loop that lets server keep interacting with both clients

Server receives message from client 1 and sends it to client 2 and receives message from client 2 and sends it to client 1.

When server receives "#" from client 1 or client 2, the connection is terminated with both clients.

SocketClient_2.java

```
import java.io.*;
import java.net.Socket;
import javax.swing.JOptionPane;

public class SocketClient_2
{
   public static void main(String[] args) throws IOException
   {
      String serverAddress = "localhost";
      int serverPort = 9090;
      Socket s = new Socket(serverAddress, serverPort);
      PrintWriter out = new PrintWriter(s.getOutputStream(), true);
      BufferedReader input = new BufferedReader(new InputStreamReader(s.getInputStream()));
      String msg="";
      msg = input.readLine();
      System.out.println(msg);
      String myname = "Kamal";
      while(true)
      {
         String toServer = JOptionPane.showInputDialog(null,"Send something");
         out.println("From "+myname+":"+toServer);
         msg = input.readLine();
         System.out.println(msg);
         if (toServer.equals("#")) break;
      }
      out.close();
      input.close();
      s.close();
      System.exit(0);
   }
}
```

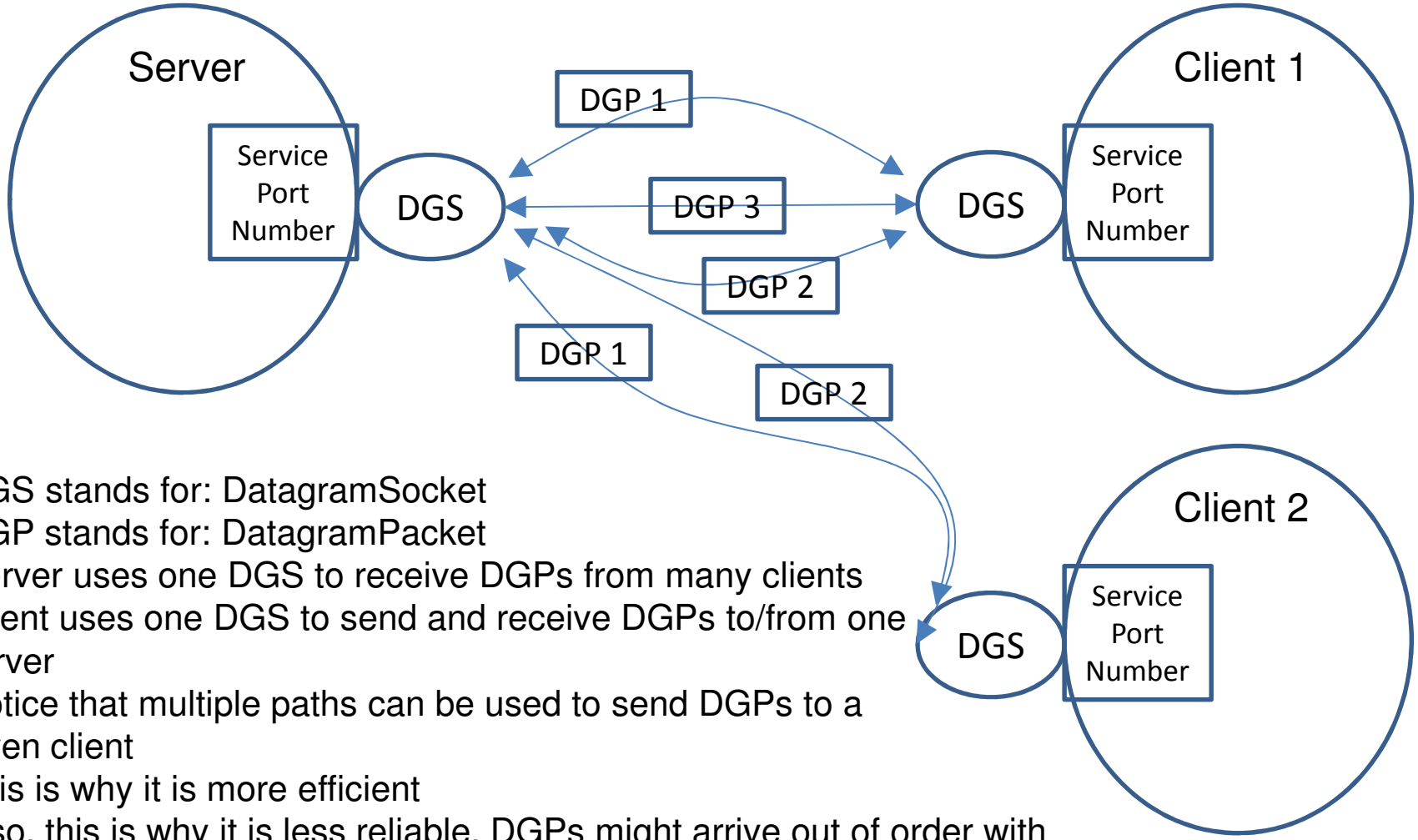
Retrieve input and output stream of server. This allows client to send/receive from server.

Client identifies himself with a name

Loop that asks client "x" to write a message. The message is sent to server and server will send the message to client "y"

If clients sent "#" to server, this means he wants to disconnect

Datagram Client/Server Application

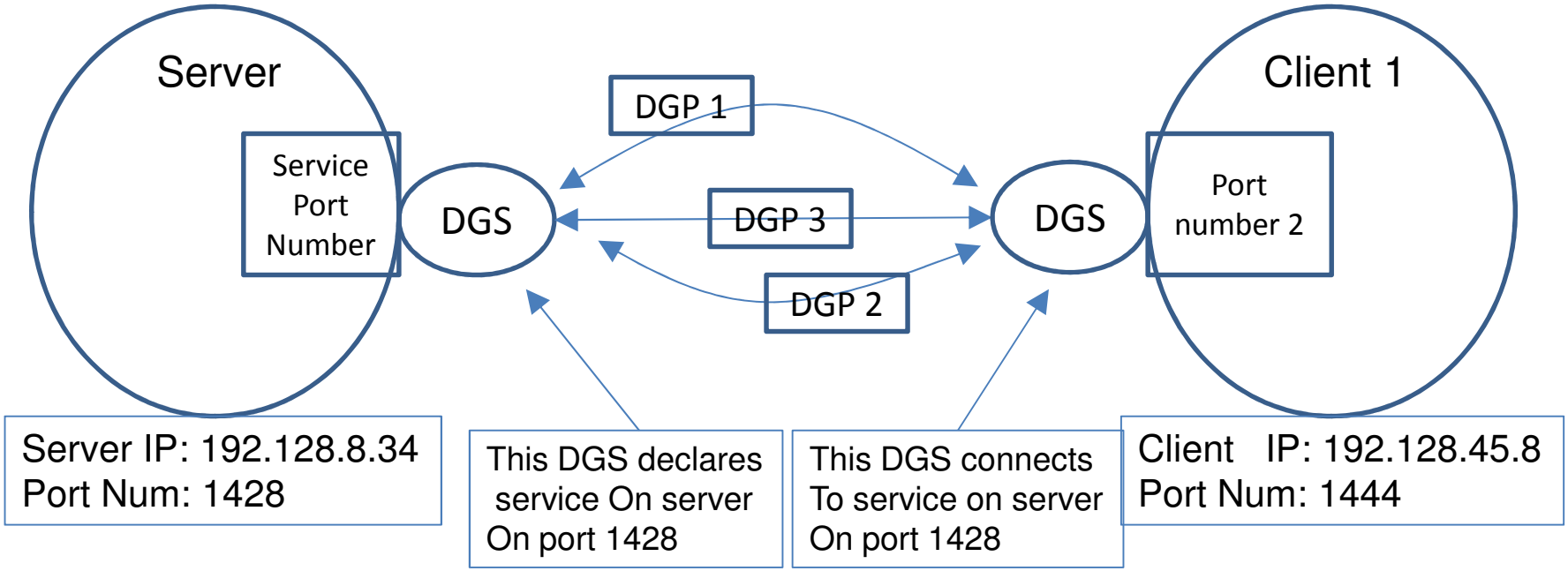


- DGS stands for: DatagramSocket
- DGP stands for: DatagramPacket
- Server uses one DGS to receive DGPs from many clients
- Client uses one DGS to send and receive DGPs to/from one server
- Notice that multiple paths can be used to send DGPs to a given client
- This is why it is more efficient
- Also, this is why it is less reliable. DGPs might arrive out of order with possibilities of duplications of DGPs

Datagram Client/Server Application

- A DatagramPacket encapsulates the following information:
  - Packet length
  - Packet data (Actual Content)
  - IP address of the host (to/from) which data is (sent/received)
  - Port number used by host (to/from) which data is (sent/received)

Datagram Client/Server Application



We have two cases:

1) Send Packet
   * In this case, you need to initialize packet with IP and port number of the destination
2) Receive Packet
   * In this case, the packet you will receive has the IP and port number of the source

Datagram Client/Server Application

Steps the server needs to do to communicate with clients:

1) Establish a server datagram socket

//This statement defines a Datagram Socket on the server and attach it to a given port number.
// This datagram socket is capable of sending and receiving data

DatagramSocket serverSocket = new DatagramSocket(port_number);

2) Send and Receive Packets

Send Packet:

DatagramPacket sendPacket = new DatagramPacket(data,data_length, destination_IP_Address, destination_port_number);
serverSocket.send(sendPacket);

Receive Packet

byte data[] = new byte[100];
DatagramPacket receivePacket = new DatagramPacket(data, data_length);
serverSocket.receive(receivePacket);

Datagram Client/Server Application

Steps the clinet needs to do to communicate with server:

1) Establish a client datagram socket

//This statement creates a DatagramSocket capable of sending and receiving datagram packets

```
DatagramSocket    clientSocket = new DatagramSocket();
```

2) Send and receive packets

Send Packet:

```
DatagramPacket  sendPacket =new DatagramPacket(data,data_length, destination_IP_Address, destination_port_num );
clientSocket.send( sendPacket );
```

Receive Packet

```
byte data[] = new byte[ 100 ];
DatagramPacket receivePacket = new DatagramPacket( data, data_length );
clientSocket.receive( receivePacket );
```

Datagram Client/Server Application

- Datagram Application:
  - DatagramServer.java: receives message from client and sends it back to him
  - DatagramClient.java: send messages to server and receive it again from him

DatagramServer.java

```
import java.io.*;
import java.net.*;
public class DatagramServer
{
   public static void main(String args[]) throws Exception
   {
      DatagramSocket serverSocket = new DatagramSocket(9899);
      byte[] receiveData = new byte[1024];
      byte[] sendData = new byte[1024];
      while(true)
      {
         DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
         serverSocket.receive(receivePacket);
         String message = new String(receivePacket.getData());
         System.out.println("Server received: " + message);
         InetAddress IPAddress = receivePacket.getAddress();
         int port = receivePacket.getPort();
         sendData = message.getBytes();
         DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, port);
         serverSocket.send(sendPacket);
      }
   }
}
```

Establish datagram socket on server

Prepare a packet for receiving data

Receive message

Retrieve IP address and port number of sender

Prepare packet and sends it to client

DatagramClient.java

```
import java.io.*;
import java.net.*;
public class DatagramClient
{
   public static void main(String args[]) throws Exception
   {
      DatagramSocket clientSocket = new DatagramSocket();
      InetAddress IPAddress = InetAddress.getByName("localhost");
      byte[] sendData = new byte[1024];
      byte[] receiveData = new byte[1024];
      String sentence = "Hi There";
      sendData = sentence.getBytes();
      DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, 9899);
      clientSocket.send(sendPacket);
      DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
      clientSocket.receive(receivePacket);
      String message = new String(receivePacket.getData());
      System.out.println("Client received:" + message);
      clientSocket.close();
   }
}
```